What's new

Displaying mathematics on the Web

Terence Tao · 29 October 2009

https://terrytao.wordpress.com/2009/10/29/displaying-mathematics-on-the-web/

tao2typ · converted to Typst 2026-10-14 (UTC)

The various languages and formats that make up modern web pages (HTML, XHTML, CSS, etc.) work wonderfully for most purposes, but there is one place where they are still somewhat clunky, namely in the presentation of mathematical equations and diagrams on web pages. While web formats do support very simple mathematical typesetting (such as the usage of basic symbols such as π, or superscripts such as x^2), it is difficult to create more sophisticated (and non-ugly) mathematical displays, such as

$$\det \begin{pmatrix} 1 & x_1 & \ldots & x_1^{n-1} \\ 1 & x_2 & \ldots & x_2^{n-1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & \ldots & x_n^{n-1} \end{pmatrix} = \prod_{1 \leq i < j \leq n} (x_j - x_i)$$

without some additional layer of software (in this case, WordPress's LaTeX renderer). These type of *ad hoc* fixes work, up to a point, but several difficulties still remain. For instance:

1. There is no standardisation with regard to mathematics displays. For instance, WordPress uses $latex and $ to indicate a mathematics display, Wikipedia uses and, the current experimental Google Wave plugins use

   $$and$$

   , and so forth.
2. Mathematical formulae need to be compiled from a plain text language (much as with LaTeX), rather than edited directly on a visual editor. This is in contrast to other HTML elements, such as links, **boldface**, colors, etc.
3. One cannot easily cut and paste a portion of a web page containing maths displays into another page or file (although with WordPress's format, things are not so bad as the raw LaTeX code will be captured as plain text). Again, this is in contrast to other HTML elements, which can be cut and pasted quite easily.
4. Currently, mathematical displays are usually rendered as static images and thus cannot be easily edited without recompiling the source code for that display. A related issue is that the images do not automatically resize when the browser scale changes; also, in some cases they do not blend well with the background colour scheme for the page.
5. It is difficult to take an extended portion of LaTeX and convert it into a web page or vice versa, although tools such as Luca Trevisan's LaTeX to WordPress converter achieve a heroic (and very useful) level of partial success in this regard.

There are a number of extensions to the existing web languages that have been proposed to address some of these difficulties, the most well known of which is probably MathML, which is used for instance in the n-Category Café. So far, though, adoption of the MathML standard (and development of editors and other tools to take advantage of this standard) seems to not be too widespread at present.

I'd like to open a discussion, then, about what kinds of changes to the current web standards could help facilitate the easier use of mathematical displays on web pages. (I'm indirectly in contact with some people involved in these standards, so if some interesting discussions arise here, I can try to pass them on.)

Comments (108)

 **jonathanfine** · 29 October 2009

Thank you for this post, Terry. I'd very much like this discussion to take place – I think it is long overdue.

I'm a member of both the Board of the TeX Users Group and the Committee of UK TeX Users Group. I've brought this request to open a discussion to these bodies, and hope for an energetic and positive response.

http://tug.org

http://uk.tug.org

Last month I organised a workshop on Technical Aspects of Mathematical Content, which addressed some of the issues in Terry's post.

http://groups.google.com/group/uk-math-content-2009/web/home

http://stadium.open.ac.uk/stadia/preview.php?whichevent=1393

There's interest in holding a similar workshop in September 2010.

 **Anonymous** · 29 October 2009

I like the concept of MathML, and there are a number of tools to get from LaTeX to it. I wish more of the major browsers supported it, and hope the STIX fonts help some. Also MathML in FireFox has some minor display issues when compared to LaTeX. I agree that I absolutely hate the inline images, there have been many times where they are too small for easy reading, with MathML you can (in the browser) set a minimum font size which greatly improves the situation. I also like that MathML allows both inline and display style math, this is essential to the stuff I tend to do (I actually just finished my custom wiki -> XHTML+MathML haskell parser (my first haskell project)). Finally, the ability to customize the mathematic display via CSS is essential (e.g. I like white text on black backgrounds) and this isn't possible with the image approach w/o rendering an image for each foreground color.

 **Anonymous** · 29 October 2009

Dear Prof . Tao,

Could you explain why the following is true, and in generall how can we figure out the order of divergence of this kind of series ?

$\sum_{n=1}^{N} \frac{1}{\sqrt{n}}$ is asymtotically is $C \times N^{1/2}$ as $N \to \infty$

thanks

 **Kareem Carr** · 29 October 2009

http://www.artofproblemsolving.com/Forum/index.php

Maybe you can sign up here and ask your question. This is probably the wrong forum.

 **timur** · 29 October 2009

You can bound the series from below by an integral.

 **Jason Rute** · 29 October 2009

This reminds me of the American Scientist article "Writing Math on the Web" . It's a good survey of the current state of web-math and the difficulties involved.



Shreevatsa · 29 October 2009

That article was from the May-June 2009 issue, and is available here: *Writing Math on the Web* (the PDF version there is better). It is also discussed at the blog of the author (Brian Hayes) here: *Math, fonts, and HTML*. I agree; it is a good survey of the state of this issue.



Anonymous · 29 October 2009

anon,
There are forums where people are happy to answer this kind of question, but it's off-topic on this blog.
Interpreted geometrically, your sum can be sandwiched between the curves
f(x) = 1/sqrt(x) and g(x) = 1/sqrt(x + 1). The area under each of these curves, as given by integrating, can be explicitly determined and is on the order of sqrt(N) in either case (if the singularity of f at 0 is appropriately avoided).



Matt Leifer · 29 October 2009

As I see it, we don't actually have a problem with *standards* at the moment, but with *implementations*. MathML is perfectly fine as a standard for displaying mathematics on webpages, but implementing it is still a bit of a nightmare. It will never be an authoring language, but there are plenty of decent ways of converting LaTeX to MathML and webpages could be easily coded to incorporate the original LaTeX markup as well as the MathML.
The problems implementing MathML stem from the requirement to serve content as appication/xml, which isn't supported by Internet Explorer, and with the need to install extra fonts. The former is not a problem if you are serving static web pages, e.g. if you just want to convert an existing paper into an HTML+MathML webpage. There are plenty of existing tools that do this well. However, it seems to me that the idea that you might want to embed MathML into content management systems, e.g. WordPress, MediaWiki, Drupal, etc. and other web-based applications was not sufficiently taken into account when MathML was originally proposed as a standard. As we have seen from the variety of mathematical blogs, this is indeed one of the most interesting ways of using mathematics on the web. Essentially, the problem is that the developers of WordPress et. al. are never going to serve their pages as application/xml because they are concerned with cross-browser compatibility and this is far more important to most users than MathML (or indeed SVG). If IE had decided to implement application/xml a few years ago, then we probably wouldn't need to be having this discussion, but it looks like they are not going to do it EVER.
The problem will be remedied somewhat by HTML5, which is going to allow MathML without any special declaration. Once all major CMSs have adopted HTML5, writing plugins that serve MathML will become much easier than it is at the moment. One action that could be taken immediately is to lobby Firefox to enable MathML in pages served as HTML5, which does not work at the moment, but shouldn't be too hard as they already support a variety of HTML5 tags.
As for the fonts problem, the STIX fonts do provide a partial solution, but still require action on the part of the user. I have seen some discussion of standards for embedding fonts in webpages, and this would be much better because webpages could then take advantage of the STIX fonts whether or not the user has installed them.



zcorpan · 30 October 2009

MathML in text/html already works in Firefox nightlies if you switch the html5.enable pref in about:config.



Qiaochu Yuan · 1 November 2009

For what it's worth, Google Chrome Frame supports application/xml.

 **Brendan McCane** · 29 October 2009

Presentation of maths on the web is important, but to me that is only a small part of the problem. Matt Liefer is right that the standards seem fine, it's the implementation that's the issue with presentation.

To me the real problem is semantics. It is no coincidence that the web really took off when a good search engine came online (alta-vista, then google). There will be little point having great amounts of beautifully formatted maths on the web if it can't be searched effectively. There are two ways of approaching this problem. The first way involves building a better search engine that for all intents and purposes searches based on meaning. Google seems to effectively achieve that without actually trying to search based on meaning – it is not clear to me that such an approach would work for maths when there are so many different representations for essentially the same constructs. The second method relies on authors embedding semantics in their markup – something like the OpenMath project perhaps. This way seems much more likely to succeed.

There is a useful analogy here perhaps with image search – most effective image search tools rely on humans to tag images appropriately. This turns out to be very effective and neatly sidesteps the problem of trying to automatically interpret images.

 **Math student** · 2 November 2009

I agree with the semantics issue, since you especially want the math concepts to be findable by a search engine for math students. I should be able to search for something like **proof:"fermat's last theorem"** or **proof:"fermat"** and have it return sensible results.

This could be allow for easy inclusion of proofs within other proofs, similar to the hypertext environment Xanadu's concept of transclusion, where you can refer to sequences of the equations of others. It strikes me that it could be difficult to implement correctly, without a deep understanding of syntax, grammar, and mathematics traditions. The semantics need to support automated theorem provers as well.

I'm concerned that too much focus on rendering issues when MathML is out there, when the existing plugin architectures for MathML should be able to handle it. (and if IE is a problem, using the Chrome/webkit rendering plugin should now be sufficient.)

Rendering is easy when the concepts are right to begin with. Using wikipedia as a large-scale testbed for these ideas should be feasible, at some point.

 **Andrew Stacey** · 3 November 2009

Adding semantics at the small-scale level is dead in the water. No-one is ever going to bother to add all that information. If I write "f: X -> Y" then it's blindingly obvious to anyone that "f" is a function, "X" it's domain, and "Y" it's codomain so why would I want to take the time to add that just to make life easier for a search engine? One could have a system for tagging papers, this was one of the ideas we floated after the recent discussion on the algebraic topology list. The key problem is always going to be persuading people to add these tags.

 **John Armstrong** · 3 November 2009

Unless f is a morphism in a non-concrete category…

 **Jerome Baum** · 24 November 2009

I guess the point here is the context. Clearly category theory isn't first-semester analysis. In a first-semester class, it is clear that a function is implied.

The question is how this relates to semantic markup. It should be possible to tag the branch of mathematics (e.g. category theory) and then interpret "f : X -> Y" automatically based on that.

> Also, how about variable search with some flexibility regarding variable naming? For example, the search ” (f : X -> Y) injective ” should also return a document containing “injective” and “g : M -> N.”



Nathan Dunfield · 29 October 2009

Apparently, Windows 7 has a new tool for converting handwritten formulae into Math ML. Here are some examples of this in action.

> 
>
> **Matt Leifer** · 29 October 2009
>
> That is so cool. You could run the output through a MathML->TeX converter and write all the math in your papers by hand. It’s almost enough to make me want to buy a tablet PC and switch back to Windows. Anyone know of similar tools for Linux or Mac?
>
> > 
> >
> > **Kareem Carr** · 29 October 2009
> >
> > There is also a feature where you can highlight incorrectly recognized parts of your equation and get a list of possible alternatives. You can see that feature in action here: http://haythamalaa.blogspot.com/2009/04/windows7-adds-math-input-panel.html
> >
> > 
> >
> > **timur** · 1 November 2009
> >
> > There is an iPhone app called DeTeXify. It is discussed here:
> >
> > > Detexify
> >
> > > 
> > >
> > > **Andrew Stacey** · 3 November 2009
> > >
> > > Detexify is great. It’s also not restricted to the iphone: it can be found at http://detexify.kirelabs.org/classify.html
>
> 
>
> **Andrew Stacey** · 3 November 2009
>
> I should look at a few of the other images of people trying to use this before getting too excited. The chemistry ones in particular were quite funny. I suppose that these will get better with training, but they’d be like speech-to-text: the amount of “misses” would always be high enough to be irritating. I suspect that I can type faster than I would ever be able to do the “draw, convert, check, modify, check again, modify again, get fed up and select from a drop-down-list of alternatives” cycle.
> Though they don’t do the conversion, I’m already finding tools like xournal and jarnal invaluable for lectures, taking notes in seminars, annotating papers, and so forth.



Matt Leifer · 29 October 2009

To clarify, there is content MathML and presentation MathML. I believe that content MathML is designed to solve the same problems as OpenMath, but I haven’t compared the specifications so I could be wrong about that. In any case, content MathML is in much worse shape than presentation MathML, since we don’t have many good translation tools and it would have to be translated into presentation MathML for display in any case.
In my opinion, trying to get authors to markup semantics as well as presentation is a losing battle because in most cases people are just trying to get their mathematical content onto the web as quickly and simply as possible. I think this is the reason why both OpenMath and content MathML haven’t really taken off, whereas presentation MathML is in comparably good shape. In fact, I am not entirely convinced that marking up semantics within the mathematics itself is an important issue. I mean, does anyone actually have any trouble looking up mathematical concepts in Wikipedia or Wolfram Mathworld as they currently stand? I don’t think so and the reason is that the meaning is

apparent from the surrounding text and explanation. That seems to provide enough semantics for most purposes. However, if anyone can think of applications for more mathematical semantics than this on the web then please enlighten me.

>  **Brendan McCane** · 29 October 2009
>
> Yes, there is a very simple application of semantic search. Say I have a particular equation or expression and I want to find similar expressions or uses or identities or everything I can about the expression. I don't necessarily know a name for the expression or theorem or may not even know which branch of mathematics it is used in. This is a very common problem for me personally (not being a real mathematician, but someone who uses mathematics a lot). Mathworld and Wikipedia are great sources for high level overviews of particular branches of mathematics, but I think there is a great opportunity for the web to become the repository for pretty much all mathematical knowledge (including latest results, obscure theorems etc). For all that knowledge to be useful, it must be easily searchable.
>
> >  **Matt Leifer** · 30 October 2009
> >
> > I think I understand that application, but I am not sure how it could possibly work. You want to type E=mc^2, for example, into a search box and get back a whole list of articles about relativity. OK, well that example would probably work at the moment because the notation is so standard, but even with semantic markup I don't see how this can possibly work in general due to large differences in notation and terminology for representing the same concept. After all, content MathML and OpenMath may be more semantic than presentation MathML, but they are not magic. They can't tell you that two things are connected if you haven't told them that they are.
> >
> > >  **Brendan McCane** · 3 November 2009
> > >
> > > Exactly the type of application I had in mind. I don't know how it would or could work either, but it seems to me like there is a better chance with semantically written maths than without. The web has been moving inexorably toward this model of semantic markup – clear separation of meaning and content from display with xml, html5 and css. I would much rather have a set of semantic elements and a standard css-like file to use, rather than a presentation type language.

 **hecker** · 30 October 2009

I am a rank novice in this area, but I have been thinking about this problem a little bit in the context of the work I do for the Mozilla Foundation (not as a developer, though I do work with them), and discussing it with Michael Nielsen in particular. Here are my quick thoughts. I don't really have proposed solutions for all of Prof. Tao's original problems, but want to add to some of the prior comments by Matt Liefer and others. Note also that though I do work for Mozilla everything I write here is my personal opinion, not an official position of or commitment by Mozilla.

1. From my admittedly biased point of view I suggest ignoring Internet Explorer with respect to these issues until/unless Microsoft is able to catch up with modern browsers like Firefox, Safari, Chrome, and Opera in terms of support for HTML5 and related standards. Again being biased, in my comments I'm going to discuss this primarily in the context of Firefox, under the assumption that Safari et.al. can implement this stuff as well if properly motivated, and if not you can always use Firefox (which of course we're happy to have you do).
2. (Presentation) MathML in HTML is in theory going to work in future versions of Firefox (post-3.5 or 3.6) once the new HTML5 parser is completely implemented and turned on by default. To the

extent this doesn't work it will be a bug and you can lobby the developers to fix things. (I know who they are so can help you track them down :-)
3. Displaying math fonts without having them installed locally is a solved (or at least solvable) problem in modern browsers, using a combination of the @fontface CSS rule (which as implemented in Firefox and other browsers supports on-the-fly downloading of fonts for arbitrary Unicode characters), quality math fonts with "web-friendly" licensing (i.e., either royalty free like the STIX fonts or with a license that permits embedded use on web pages at relatively low cost), and (optionally) the Web Open Font Format (which reduces the bandwidth needed for on-the-fly downloading by supporting compressed fonts). (WOFF support is not in Firefox yet but is coming, and in any case is just an optimization.)
4. Modulo any bugs, items 2 and 3 together should enable reasonable display of math (expressed as presentation MathML) in a manner that is web-native, in the sense that it "just works" when you go to a particular web page, doesn't require hacks like displaying character images, can be styled using CSS (e.g., for color, font style, size, etc.), can be manipulated via JavaScript (i.e., through operations on the HTML Document Object Model), and in general can be dealt with by web applications (whether client or server based) similar to how vanilla HTML is today.
5. On the input side I can see a variety of ways to approach the problem. One approach is to continue to use LaTeX as an input format and do on-the-fly conversion to MathML for display. This conversion could be done server-side (which is how I understand it's done with the LaTeX plugin for WordPress) or could be done client-side using a JavaScript library downloaded with the page (similar to what the jsMath library does, but with conversion to MathML).
Given the advances in JavaScript performance in modern browsers I suspect that client-side on-the-fly LaTeX-MathML conversion is going to be increasingly feasible, so that one could imagine a purely browser-based web application (i.e., no server-side support required) where you can type in LaTeX and get near-instantaneous conversion to MathML and display using math fonts. (I'll show my ignorance here — does any such application exist today, even in prototype form? If so I'd be interested in hearing about it.)
6. For those who (for whatever reason) don't want to use LaTeX, I suspect that it would be possible to construct a client-side JavaScript-based equation editor that would (again) generate MathML on the backend for display, analogous to the rich-text editors you see implemented in various blog software products and content management systems using JavaScript-enabled dynamic HTML techniques. (Again, if anyone knows of existing work in this area I'd be interested in knowing about it.) Such an equation editor could presumably also convert MathML to LaTeX for export, using similar techniques to those that I understand are implemented in other contexts.
7. Moving into more blue-sky territory, once we're fully in the world of HTML5 and associated technologies there are lots of interesting things that could be implemented. In particular the HTML5 canvas element allows arbitrary 2D (and, in future, 3D) drawing using JavaScript, which can then be combined in various ways with SVG (including animation of SVG images), video (using the HTML5 video tag), and regular HTML (including, in future, MathML). (The hacks.mozilla.org site is a good source of demos and information on advanced techniques in this area.)
For example, Mozilla Labs has used canvas to implement Bespin, a web-based editor for programmers (including version control). We also have some students and other volunteers using canvas in the "Processing for the Web" project to do a complete web-based implementation of the Processing language and associated development and runtime environment used by artists and people doing data visualization.
One can imagine these technologies also being used to create advanced web-based applications for mathematics and mathematicians, for example some sort of shared whiteboard for network-based collaboration, perhaps eventually backed up by server-side mathematics engines derived from open source technologies like those in Sage and related products.

Of course, none of this exists today, and even the relatively simpler things (like dead-simple use of MathML in HTML plus downloadable fonts) will have to wait on future versions of Firefox and other browsers. However HTML5 and related technologies are coming, even perhaps eventually to IE, and I don't think it's too soon to start thinking about what sorts of applications might be possible, and even to begin prototyping them where possible.
Mozilla's role in all this will be as a supplier of browser technologies that could form the infrastructure for such applications. However we certainly have an interest in knowing about what sorts of applications people would like to see, and what people would need in the browser to make them possible. We could also potentially support (to some extent or other) particular third party initiatives to implement such application. For example, as part of our Mozilla Education initiative we are helping recruit programming students at various universities to work on the Processing for the Web project I mentioned above, and providing them with access to Mozilla developers where needed. I could see us possibly doing something similar in this area if there were people out there interested in working on it.



Matt Leifer · 31 October 2009

Nice to hear from someone in the Mozilla camp about this. I know of one javascript LaTeX->MathML converter, which is called LaTeXMathML. I tried to write an "on-the-fly" LaTeX editor using it, as you described, but it was a nightmare because the script is very buggy and the code is a mess. We really need a new implementation. Somewhat bizarrely, LaTeXMathML works with pages served as text/html already on Firefox because there seems to be a difference between converting client-side and serving a page. Somehow, the browser is "tricked" into displaying the MathML, even though it shouldn't do it according to the HTML4 specification. Strictly speaking, I guess this is a bug, although it is a feature from our point of view. There is also jsMath, which does a similarly good job of client-side mathematics conversion, although it is not using MathML. The main point is that we have had decent javascript technologies for mathematics for quite some time, and yet we have not seen the kind of rich web-applications involving math that you describe. I think the main reason for this is that the people who care about math on the web are largely academics rather than hackers and don't have the time or inclination to learn the full stack of web technologies, i.e. maybe they learned a little php in order to hack a WordPress profile, but no javascript/AJAX skills. Therefore, we do need help from experienced web-developers to get these sort of projects off the ground.
One thing I would like to see is a comprehensive collaborative web-based LaTeX authoring system. I don't think it would be too hard to implement this with something like Bespin. What would be needed is:
– LaTeX syntax highlighting.
– Ability to compile a LaTeX document server-side and download the resulting pdf.
– Ability to switch between LaTeX source and a preview that uses web-scripts to convert to a HTML+MathML representation.



hecker · 31 October 2009

Your point that we haven't seen rich web-based math applications of the type I described is well-taken, and I agree with your explanation. To me at least this seems very similar to the situation prior to our starting the "Processing for the Web" initiative I mentioned above: It was something that could be of benefit to digital artists, but most digital artists didn't have the programing expertise to implement it, and the people who did have the expertise were focused on other things. Certainly Mozilla itself had no interest in devoting any significant staff time or funding to the problem.

The way we addressed this dilemma in the case of Processing for the Web was to recruit students to do the bulk of the work, under the auspices of our Mozilla Education initiative wherein we work with faculty at various institutions who teach programming courses and are interested in having their students work on open source projects. However we also needed (and were able to get) two additional things, namely a clear and achievable goal in terms of what we wanted to implement, and a person who knew both the Mozilla technologies (JavaScript, Canvas, etc.) and the subject areas in question (Processing, graphics, digital art, etc.) and could lead and coordinate the work of the students.
We also had moral support from leading figures in the field (Ben Fry and Casey Reas, inventors of Processing and prominent digital artists) and from people within Mozilla (who were interested in seeing Mozilla technologies showcased in this way) but IMO these were secondary to having a clear and compelling technical vision and strategy and a strong technical lead with a foot in both worlds.
If we had a similar fortuitous combination of plan and person for web-based math applications using HTML5-related technologies (HTML5+MathML, JavaScript, CSS, etc.) then I'd be willing to explore the possibility of doing something in the Mozilla Education context similar to what we're doing with Processing for the Web — basically a Mozilla-endorsed project where we'd provide some coordination help but no funding or dedicated staff time. I can't guarantee that anything at all would come of this, but it might be worth at least doing some thinking and discussing around the idea.



Jacques Distler · 31 October 2009

"LaTeXMathML works with pages served as text/html already on Firefox because there seems to be a difference between converting client-side and serving a page. Somehow, the browser is "tricked" into displaying the MathML, even though it shouldn't do it according to the HTML4 specification."
It's not really a bug (or even a Spec violation). Mozilla has no compunction about rendering MathML inserted into the DOM (via Javascript). But, except in html5-mode, it won't **construct** a DOM containing MathML, from a text/html document.
(This is obvious: there's no notion of namespaces in HTML, but there is in the DOM. To render correctly, what we need are MathML elements, in the MathML namespace. In HTML4, there's no way to get such elements into the DOM, by parsing a document. The only way is via DOM-scripting. HTML5 sidesteps this, by special-casing MathML and SVG, allowing those to be placed, directly in the DOM, in the correct namespaces.)



anonymous · 30 October 2009

A nice math-package for all browsers that unfortunately seems to be not known well enough is jsMath (see http://www.math.union.edu/~dpvc/jsMath/).



Vincent Beffara · 30 October 2009

I was just about to mention jsMath, which is absolutely great. It is able to scan the usual TeX/LaTeX math markup (both dollar-signs and backslash-parentheses, both inline and display), meaning that in many cases copy-and-paste from a .tex file will work out of the box. It can use native true type versions of computer modern (available for download), or images if these are missing.
Shameless plug : we use it (and are extremely happy with it) for the "images des mathématiques" website at http://images.math.cnrs.fr/ within a CMS (spip) and being able to just copy and paste opens us to many more contributions from "old-style" mathematicians … have a look there to see how it looks in real life !

 **Anonymous** · 30 October 2009

I'm using jsMath too! I use it with Tiddly wiki as a neat personal wiki with math input. I tried wikimedia and other wiki softwares before, all require compiling the input to pictures, which was kind of un-handy. With jsMath it's much simpler! All the math are just font that can be used directly.

 **Jacques Distler** · 31 October 2009

I'm dubious about the whole client-side LaTeX → MathML idea.
1. jsMath has real perfomance issues on math-heavy pages (though it works *marvelously* on pages with a limited amount of mathematical content). I expect other client-side solutions will have similar issues.
2. I don't think LaTeX is a good wire-format for the web.
3. It doesn't address the main issue for clients, today, namely fonts (which you will still need).
4. In those instances where you want dynamically-updated mathematical content, I don't see any telling advantage over server-side LaTeX → MathML + AJAX.
5. In many instances, you still might want to be able to provide a fallback to PNGs. Even the *horrid-looking* ones that are available here are better than nothing. LaTeX → PNG is an even less likely candidate for client-side handling.

 **hecker** · 31 October 2009

I'll address your comments in rough reverse order. First, I'm proposing doing something innovative which would enable new uses for math on the web (and/or old uses in new contexts); if there are systems today that are good enough for their current uses then there's not much advantage in trying to replace them wholesale, which (among other things) would entail trying to handle all the legacy cases (old browsers, no fonts, etc.). Thus I'm going to assume for purposes of this discussion that we're dealing with Firefox and other modern browsers that have similar feature sets.

With regard to using LaTeX or not, I think that really depends on the context and the users. In some cases we'd be dealing with MathML generated in other contexts, in some cases might want to have AJAX-enabled equation editors that generate MathML, and in some cases might have to deal with LaTeX as the preferred input format. You folks know much better than I which types of applications would be of most interest, and how you'd like them to work.

With regard to client-side vs. server-side, in the case of Processing we had no choice but to implement a client-side solution because we had to support rich animation and other features where you don't want to make a round-trip to the server. In the case of displaying math the requirements aren;t quite so rigorous, so a server-side solution (perhaps AJAX-enabled) might make sense in many contexts. However I wouldn't want to rule out doing stuff client-side where needed.

With regard to the efficiency of doing things like client-side conversion of LaTeX to MathML, it's not 100% clear to me where the slowness of systems like jsMath arises. Part of it might be that it's having to do extra work exactly because it's not generating MathML (e.g., in doing positioning of glyphs), part of it might be due to conversion of LaTeX to MathML being inherently inefficient due to the algorithms required, and part of it might be the slowness of executing JavaScript in the browser.

The last problem at least is one that all of the browser vendors are addressing to one degree or another, with some innovative techniques to optimize JavaScript execution to speeds near that of native code. The first problem (to the extent it exists) would go away if the only task were to generate MathML and we could assume adequate fonts were available (whether pre-installed or downloaded on the fly).

 **Jacques Distler** · 31 October 2009

"Thus I'm going to assume for purposes of this discussion that we're dealing with Firefox and other modern browsers that have similar feature sets."
For present purposes, that means Mozilla-based browsers. It eliminates IE+Mathplayer, and it eliminates otherwise-modern browsers without native MathML support (Webkit and Opera).
I'm fine with that, but let's make sure we know what we're discussing.
"With regard to using LaTeX or not, I think that really depends on the context and the users."
Sure. For the people Terry Tao is addressing, LaTeX input is as natural as breathing. But others will find authoring in LaTeX as forbidding as MathML.
"With regard to the efficiency of doing things like client-side conversion of LaTeX to MathML, it's not 100% clear to me where the slowness of systems like jsMath arises."
You're absolutely right that jsMath does much more complicated things (it's, among other thing, a whole rendering engine, written in Javascript) than a javascript LateX→MathML converter would. So perhaps that comparison is unfair …
One advantage a client-side LateX→MathML converter could offer is is that it could be used to support (with minimal changes) content such as appears on this blog. Here you have server-side conversion of LaTeX→PNG, with the original LaTeX embedded as @alt attribute of the resulting <img/>. For clients that support MathML, the client-side converter could grab the content of the @alt attribute, convert it to MathML and replace the <img/> with the resulting <math> element.
That's in addition to whatever other cool applications you build around it…

 **Terence Tao** · 1 November 2009

Incidentally, it appears that Chrome does not currently support MathML either (an odd flaw in an otherwise decent browser). Presumably this is the type of issue that will solve itself over time, though.
Ideally, LaTeX should just be one of the ways to input mathematics; I'm sure there will be other ways also (e.g. Windows 7's handwriting recognition tool). What I would like to see is a way to type a string of LaTeX, press a button or hotkey analogous to, say, the "Make boldface" (CTRL-B) or "Create hyperlink" (CTRL-K) buttons/hotkeys standard on most visual editors, and transform that string into an editable (and cut-and-pasteable) piece of MathML (or other suitable format). (One might also set an option to automatically convert $text$ directly into MathML, much as many editors automatically convert _text_ into *text*, *text* into **text**, etc.) Of course, this is not so much a standards issue (except insofar that MathML is not widely supported currently) as an issue of getting a decent visual editor…

 **Jacques Distler** · 1 November 2009

"Incidentally, it appears that Chrome does not currently support MathML either (an odd flaw in an otherwise decent browser). Presumably this is the type of issue that will solve itself over time, though."
More precisely, WebKit doesn't support MathML, so none of the WebKit-based browsers (Safari, Chrome, …) does. Adding MathML-support to WebKit would make a nice Google Summer of Code project, if someone at Apple or Google were willing to sponsor it.

"What I would like to see is a way to type a string of LaTeX, press a button or hotkey … and transform that string into an editable (and cut-and-pasteable) piece of MathML (or other suitable format)."
Easily done with itex2MML. The AbiWord word processor uses itex2MML (and GTKMathView) as the back-end for its equation editor.



hecker · 1 November 2009

Yes, you're correct, my (admittedly selfish) focus is on Firefox. My hope would be that if there were compelling new applications that leveraged MathML as well as the standard set of HTML5-related technologies, it might serve as a spur to getting MathML support at least in Webkit and thus in Safari and Chrome, and perhaps in other browsers as well. Based on the discussion thus far, it seems as if it might be worth looking at the feasibility of a streamlined LaTeX-to-MathML converter written in JavaScript and suitable for use in client-side web applications, perhaps building on the existing LaTeXMathML package or implemented from scratch.
I'll go back and talk to some other Mozilla people and see if there's any interest in doing something like this in a way that could be packaged as a student project (or set of them) as part of our Mozilla Education activities. (It could be positioned as a Google Summer of Code project as well, of course.)



Jacques Distler · 1 November 2009

"My hope would be that if there were compelling new applications that leveraged MathML as well as the standard set of HTML5-related technologies, it might serve as a spur to getting MathML support at least in Webkit…"
Actually, at this point, I suspect the converse is more likely to be true. Getting MathML support in Webkit would be an enormous boon to the acceptance of MathML generally, and to the popularity of any such new web application, in particular.
Alas, that's not something that you, or the Mozilla Foundation are likely to have much direct impact on…
"Based on the discussion thus far, it seems as if it might be worth looking at the feasibility of a streamlined LaTeX-to-MathML converter written in JavaScript and suitable for use in client-side web applications …"
I would humbly suggest looking at itex2MML as a baseline (in terms of feature-set) for such a project. One could either do a from-scratch implementation, or translate Lex/Yacc to JS/CC, and build the parser that way.



hecker · 1 November 2009

"I would humbly suggest looking at itex2MML as a baseline (in terms of feature-set) for such a project."
If we do decide to go forward with some sort of *TeX-to-MathML project in the Mozilla Education context (and, again, I make no guarantees on that), we'll definitely take a look at itex2MML.



Michael Whapples · 31 October 2009

One issue about maths on the web which hasn't been discussed in this is the accessibility of maths. The only standard/system of putting maths on the web which is properly accessible is mathml. I will be honest and say I hadn't heard of openmath and so cannot comment on its potential for accessibility at this moment, I do intend to look at it.

When I am talking of accessibility I am talking from the view of a visually impaired person and so I don't intend to represent any other disabilities, however others should not be ignored, it would be best if someone knowledgable of the other needs were to put their case.
What makes mathml so good? While take up has been slow for mathml and really more software should support it, it is one of the few ways of displaying maths which do have accessible implementations. The main piece of software I will discuss is mathplayer (I personally prefer firefox but firefox doesn't give the mathml accessibility available with mathplayer). As some of you have said, images of the equations can be hard to see, don't always look correct, etc, imagine what it is like when you can't see too well and possibly want a specific font size and colour scheme. Mathml does allow this sort of alterations. In the case of someone who can't see at all, then mathml and mathplayer allow speech output, you should be able to try this out for yourselves. As well as the in built speech output in mathplayer, mathplayer will allow communication of what should be spoken to screen reader software (software which will speak the contents of the screen, commonly used by visually impaired people). Mathplayer isn't perfect but it certainly proves that mathml holds the information needed to make maths on the web accessible.
Some here have been discussing dynamically generated pages, please make as much as possible server side. Why I say this is although client side generation of content can be accessible when done correctly, client side processing does hold a much greater risk of being inaccessible to screen readers due to how closely they have to work with the browser to extract the information.
Now why I say do NOT use images. As I said earlier there is the problem of not being able to customise the colours, font size, etc when using images, the lack of this can be a big problem for people with low sight. Also an image gives no clues as to what it shows. The alt tag in HTML can be used but this is not required and may not be filled in usefully even if it is used. Due to this an image reveals nothing to those with with no sight and so relying on screen reader software. If you really must use images then please do the best thing under that situation and give the equation's LaTeX source as the image alt tag.
OK, that's probably enough from me at the moment, you probably have heard enough for now. I will give more information it is wanted though.



hecker · 31 October 2009

We're quite familiar at Mozilla with the problem of making advanced web applications accessible (we've been funding and otherwise supporting web accessibility work for several years now), and I agree with your point about getting away from image-based systems and to systems where there's at least a hope of making them accessible. To my knowledge we haven't done much if anything in the area of accessibility of MathML, primarily because the low rate of adoption of MathML made it a much lower priority than other accessibility issues we had to address in Firefox. However if we were to endorse some sort of MathML-related project like I discussed above then we could definitely go back and look at any remaining accessibility issues.
With regard to client-side vs. server-side again, one of the things we spent a lot of time on was promoting and implementing support for the W3C WAI ARIA specification, which was specifically intended to allow dynamic web applications (i.e., using advanced client-side techniques) to be made accessible. Firefox currently has the best support for WAI ARIA of any browser, and has good support from screen readers such as JAWS, NVDA, and Orca that support WAI ARIA. So I think making a MathML-based web-based math application accessible is a solvable problem, at least as far as Firefox is concerned.



Michael Whapples · 31 October 2009

May be my comments on client side processing were slightly unfair. When client side stuff is done correctly it can be perfectly accessible and equally if server side stuff is done poorly

then it will be inaccessible. I just feel that sometimes client side code can tempt developers into inaccessible interfaces easier than server side code does.
As for mozilla accessibility, yes I know what it can offer and I know some of the challenges faced to make really good access to screen readers when dealing with complex information such as equations. While mathplayer is good in what it does and proves that mathml is up to providing access to maths, it doesn't offer good navigation of the equation should the user need to step through it bit by bit (mathplayer basically passes a string of what should be said to the screen reader and so the screen reader is unable to jump to certain parts of the equation, eg. the right hand side). I don't believe this is due to a lack of willing from design science to provide the features but rather that the accessibility APIs and screen readers aren't designed to accept math structures. I think to get this moving it may require one part of the chain of accessibility to make a brave leap and try and provide decent math navigation (I don't mean to pressure mozilla but it would be nice if firefox got this and may be orca or NVDA were also altered to work with mathml in firefox, may be its because in the past some of the commercial screen reader producers have said that maths isn't worth their time as not enough users would want such features). I would try and help out with any opensource project attempting to make maths on the web more accessible.



hecker · 1 November 2009

I looked briefly at the accessibility features of mathplayer. They use MSAA as the accessibility API, which is not as complete an API as either IAccessible2 (which is what we use for Firefox on Windows) or UI Automation (the new Microsoft accessibility API for Vista and Windows 7). This may account for some of the missing features.
"I don't mean to pressure mozilla but it would be nice if firefox got this and may be orca or NVDA were also altered to work with mathml in firefox, …"
If there are specific accessibility bugs you have encountered with MathML in Firefox, or new feature requests you'd like to see implemented, please feel free to file a bug against the MathML component and copy me (hecker@hecker.org). I'll make sure that the right people are notified of the bug reports.



Aaron Leventhal · 5 November 2009

When I still worked on Mozilla accessibility, I did some blogging on this topic and even starting a Google group mailing list called free-math. The idea was that if we could create some open APIs, implement it in Firefox and NVDA/Orca, the commercial products would eventually have to follow us for feature parity.
You won't find disagreement that math is very important. IMO math is a basic kind of information and support should be built into the core products, not some special tool you need to go find.
While doing research for the mailing list I found that there are numerous open source libraries for converting math to Braille. There is been little or no coordination between projects, which could be seen as a waste of resources.
Despite these issues, the positive side is that there is code for converting TeX or MathML to Braille. Text to speech would probably need to be built right into the AT for the most streamlined experience (at least that's the understanding I had after discussing with Mick and Jamie, I could be forgetting something there). All we really need though, is for some decent coder to get in there and start trying to hook things up. Wikipedia has a ton of math that could be made accessible in NVDA right now, if someone spent a couple of weeks hooking up one of the Braille libraries and some basic TTS for it. It could be polished later.

However, while there was some interest there was very little follow-up by the community. The problem seems to be there are few people working on the open source accessibility stuff. For example, Mick and Jamie have their hands completely full making NVDA work with Windows 7, Adobe products, Web 2.0, etc.
But, if we're going to get to a feature like accessible Math any time soon, we need more contributions from the community to open source accessibility products. I believe in approach and/or individual contributor showed a lot of potential, there would be possibilities of getting funded for the work. But, no guarantees.



hecker · 5 November 2009

Some context for other folks: Aaron Leventhal is the person most responsible for the (relatively good) state of Mozilla/Firefox accessibility today. NVDA is an open source screen reader for Windows; the lead developers are Mick Curran and Jamie Teh. The NVDA project has received funding from Mozilla, Microsoft, Adobe, and others. Given its open source nature NVDA would be a good platform to test out new ideas about making mathematics on the web more accessible. (This is true for Orca as well, though being Linux-based limits Orca's potential user base.)
I should also add that Firefox 3.6 beta 1 can be configured quite easily to turn on the HTML5 parser and test the use of MathML (and SVG) in ordinary HTML documents. I encourage anyone with an interest in MathML and accessibility to download the Firefox 3.6 beta and file bugs you might find relating to MathML and accessibility.



Jacques Distler · 5 November 2009

In some ways, it's very odd that there hasn't been more work on this in the (admittedly small) open-source accessibility community.
I hope this conversation spurs some action.
"I should also add that Firefox 3.6 beta 1 can be configured quite easily to turn on the HTML5 parser and test the use of MathML (and SVG) in ordinary HTML documents."
I've noticed.
But there's not a lot of content out there that immediately lends itself to testing. Most sites, which support MathML/SVG, send application/xhtml+xml to compatible browsers. That's the way you HAD to do it before now.
So, if you went, say, to my blog in the new Firefox beta, you'd see the MathML OK, but you wouldn't be testing the HTML5 parser; you'd be looking at the output of the XML parser.
I'd be happy to fix things (send text/html to browsers with the HTML5 parser turned on), but I don't know how to detect that.
P.S.: As I noted on my blog, work has started on MathML support in WebKit.



Michael Whapples · 5 November 2009

Aaron, taking a look of where I am at now, possibly I could do some stuff on math accessibility. I admit I certainly have my limitations, I only know the python and java programming languages (python is good here as both NVDA and orca are written in python) and I also have managed to do some interfacing with C from those, cython/pyrex in python and JNA (Java native access, http://jna.dev.java.net) but I don't know C or C++ themselves. I have recently gained interest in liblouisxml as it now has UK Braille maths support, however I don't really understand how to write tables and such for it.
So I would say I am someone with an interest in this and a personal motivation to get it done and I know some of the stuff to get it done. However I may need some support with

learning the extra bits and pieces and also may be to guide me through it (its probably quite a move from some of the things I have done in the past).
So if anyone would like to take me up on this and help me with getting it done then feel free to contact me.

 **Jacques Distler** · 31 October 2009

To add a little to Michael's comment on accessibility:
1) MathPlayer, the MathML plugin for InternetExplorer has accessibility features. With a screen-reader installed, it will read MathML equations aloud (and, having heard it in operation, I can say that it does a quite creditable job of it. It can also convert MathML equations to braille.
2) While most accessibility discussions are geared to the needs of blind users, there's a much larger number of users who are not blind, but suffer from various degrees of visual impairment. For them, simply rescaling the text-size is what's needed. But PNGs of LaTeX text, as created by tools like LaTeXRender (used on this blog), rescale **very poorly**, and are nearly illegible at 200% or 500% magnification. MathML, like any text, rescales just fine.
3) I hadn't really thought about the additional burden that client-side conversion would add to the screen-reading user. Perhaps that's another good reason to prefer server-side (or server-side + AJAX) rendering. Of course, you need to be careful with any use of client-side scripting (including AJAX), to make it accessible.

 **jonathanfine** · 1 November 2009

With mathematical content there are local solutions, but often they do not fit together to produce a global solution. For math on web pages there are at least 3 major methods, namely images, mathml and jsMath. Each has its own advantages (works anywhere, accessibiilty, display quality for example). And then for print and on-line PDF there is LaTeX.
The difficulties in copy-and-paste is a major indicator of the lack of a global solution. If we could write in LaTeX and translate reliably into XML/HTML for use as web pages it would be very nice. If we could convert XML content in TeX/LaTeX for print and PDF that would also be nice. And if all browsers had TeX typesetting built into their rendering engine, and came with TeX's math fonts, that too would be very nice.
Bringing about a global solution will necessarily require changes in the local solutions. For example, would authors be willing to accept a restricted form of TeX macro language? (Did you know that you can program the lambda calculus using just TeX macro expansion? [Alan Jeffry, lambda.sty]) Or even no macros at all (because web pages don't provide macros)?
My personal view is that web services are a good way of taking things forward. If there are reliable means of translating TeX mathematics to MathML and vice-versa, and either to images, make them available as a web service. Google charts is a good example of how effective a web service can be.
(By the way, it seems that Google have introduced their own TeX-like language as an undocumented feature of Google charts.)

 **Jacques Distler** · 1 November 2009

[I]f all browsers had TeX typesetting built into their rendering engine, and came with TeX's math fonts, that too would be very nice.
The web is fundamentally incompatible with the assumptions of TeX typesetting. The idea of building TeX rendering into browsers was considered and rejected long ago. (Though it keeps cropping up, from time-to-time, as here.)
As to fonts, the Computer Modern Fonts are not Unicode compatible, and so need to be kept as far away from the browser as possible. Their Latin Modern counterparts *are* Unicode compatible.

But, despite being excellent print fonts, Computer/Latin Modern are less-than-ideal screen fonts. Hence the lack of popularity of Latin Modern for screen/web use.
The STIX fonts provide a comprehensive set of mathematical glyphs that *do* work well on-screen. Coupled with suitable text fonts, they should be more than adequate.
"If we could write in LaTeX and translate reliably into XML/HTML for use as web pages it would be very nice."
You need to look at Gellmu.
"If there are reliable means of translating TeX mathematics to MathML and vice-versa, and either to images, make them available as a web service."
Copying and pasting equations to/from a web page, offering such a conversion, would get *really old* really fast. Offering an API for the conversions would pass the "not completely annoying" test. But, seriously, converting some restricted dialect of LaTeX to MathML is not rocket science. Designing a web application to use such an API is no easier than using the above-mentioned (but still hypothetical) javascript on client-side, or using something like itex2MML on the server-side.



Matt Leifer · 4 November 2009

Gellmu is really very nice, but it is a bit off-putting for people who are just looking for something to plug into their web-app. For example, I wish someone would come up with an implementation in a language that is commonly used for web programming, i.e. not Emacs Lisp, and I wish someone would write some documentation designed for end users that doesn't assume a lot of knowledge about SGML and LaTeX (or at least doesn't assume that the reader wants to wade through a lot of technical discussion about the SGML philosophy of Gellmu).



Jacques Distler · 1 November 2009

Heck! If you wanted to be fancy, you could stick an HTML <img> in the <desc>element, to provide fallback to a PNG.



smoov · 1 November 2009

..or you could all just get Google accounts and copy/paste your equations from Google Docs. It's really that simple.



Andrew Stacey · 1 November 2009

You are Larry Page and I claim my five pounds.



Jacques Distler · 1 November 2009

If all you want is a Web Service which produces PNG pictures of LaTeX equations, Terry's blog (or, more precisely, WordPress.com) already provides one.
All you need to do is percent-encode the LaTeX equation as the query-string of a URL like this one or this one.
Does WordPress.com care if they're used as an equation-conversion service? Presumably not, as they don't bother to check the HTTP-Referer string.
Anyway, as I've already said, I think these PNGs look crappy (particularly when magnified). They're inaccessible, … etc, etc.
But there's no point in discussing the need for a Web service to produce them — that's a done-deal.



Randall Farmer · 1 November 2009

I've set up a bit of JavaScript that scans through a page and makes images for LaTeX enclosed by dollar signs, double dollars, or … or

…

. Nothing groundbreaking but it makes it easy to use LaTeX in Web apps that don't have native support or plugins. It also supports uploading a .sty file with simple macros and commands.
It's at http://mathcache.appspot.com/ . (It's public domain; please hack on it.)
Reactions to Terry's points — partly judging my script against them, partly overall thoughts:
Standardizing on delimiters: It seems like the biggest win to standard delimiters is that automated tools like indexers can find the LaTeX and treat it specially. It also helps with cut and paste. On the other hand, it's not that hard to tell a computer to parse a soup of different delimiters. For example, it wouldn't be so hard to support WordPress-style and MediaWiki delimiters in my script. (That's not a promise that I'll add that, but do speak up if it'd be useful to you.)
Visual editing: I haven't done anything to enable visual editing. I do have some bookmarklets at the site to help you generate previews of your rendered LaTeX easily in rich-text editors like GMail's and WordPress's. mimeTeX, a fast partial TeX implementation, could generate a "live preview" off to the side of the LaTeX snippet you're editing even more quickly. (jsMath too.) Combining fast preview with something like MediaWiki's editor — where friendly buttons insert the text needed to mark up different constructs — could make editing math easier for LaTeX novices and mortals.
Cut and paste: My script makes cut and paste work like WordPress. The delimiters are part of the copied text, too, so you could copy math into another document using my script and it'd still be rendered as math.
Converting whole documents and sections to and from LaTeX: Yes, work to be done to make this seamless — not much more to add there. It seems like full TeX documents, with all of their global document state and macro features, need a substantially different approach than snippets.
Implicit in all my replies is that I don't think we'll get help with this from major browser vendors, especially the one in Redmond. To that extent, we're on our own creating the server-side infrastructure and slick UIs that we'll be using to work with math on the Web in the future.



Randall Farmer · 1 November 2009

I left out one of Terry's points:
Downsides of TeX as static images: mathcache.appspot.com can be told to produce images for pages with light or dark backgrounds, and for what it's worth the updated static image is generated implicitly when the webpage changes; the user doesn't have to do anything special. Re: resizing, LaTeX-based images certainly aren't resolution-independent, but neither is much of the web! (We could render them at a higher res. to help, or produce SVG for browsers that support it.)
I should also give mad props and due credit to John Forkosh, who provides a public TeX-rendering service that my script uses and wrote the mathTeX rendering script and mimeTeX partial TeX implementation.



Andrew Stacey · 1 November 2009

Why not just make the backgrounds transparent?
Over at the n-forum, we use images for mathematics and get them from the wordpress server. The script (which is a simple adaptation of the usual wordpress plugin) simply sets the background colour of the resultant PNG to transparent. Then when the user changes their colour scheme, at least the background matches even if the text colour doesn't.



Randall Farmer · 2 November 2009

Good question — the answer is IE6 support. It takes additional code to make transparent PNGs work there, and I do want the script to work there.



Andrew Stacey · 2 November 2009

I don't understand this. Does IE6 not display transparency correctly? Does it simply ignore the alpha channel? If so, there's no problem: choose a reasonably standard colour for the background and set the alpha channel accordingly. Then anyone (*(&^Y#$ enough to be using IE6 will get the "standard" background and anyone using a browser that can display PNGs correctly will get the transparent background.



Randall Farmer · 2 November 2009

IE6 gives transparent PNGs a gray background if you don't use the AlphaImageLoader hack, MS says. If you don't care, you can put '\png' in the window.mathPreamble variable.

Implementing the hack that fixes IE's behavior and making PNG the default format seem like reasonable things to do next time I'm throwing some spare cycles at the script.



Randall Farmer · 2 November 2009

Realized there's a really trivial way to maintain backwards compatibility: have IE 6 ask the server for GIFs, everyone else ask for PNGs. Done!



Jacques Distler · 1 November 2009

"Re: resizing, LaTeX-based images certainly aren't resolution-independent, but neither is much of the web!"

Perhaps. But these PNG pictures of anti-aliased fonts seem to rescale particularly poorly (and, frankly, look blurry and indistinct, even at their "design" resolution).

"We could render them at a higher res. to help, or produce SVG for browsers that support it."

You could produce SVG instead?

Wow! **Do it!** That would look **so much better**.

(Put the original LaTeX in the <desc> element, for those without SVG support in their browsers.)



Randall Farmer · 2 November 2009

Jacques: Sorry, I didn't draw a clear line between far-out speculation and the stuff I could easily rig up. SVG output is hard. If anyone makes a server-side script like Forkosh's MathTeX or LaTeXrender that outputs SVG, I could rig up the caching and client-side stuff to show raster images to IE and SVG to browsers that support it.

(Because my script finds and replaces LaTeX on the client side, it can do different stuff for different browsers.)



Andrew Stacey · 2 November 2009

There are MathML to SVG converters in python and in java. The python one is called SVGMath, the java one is pmml2svg. Both are at sourceforge: http://sourceforge.net/projects/{svgmath,pmml2svg}. I don't know if those will be any use to you.



Randall Farmer · 2 November 2009

Want to emphasize that this is NOT a feature promise, but this looks really interesting — LaTeX, GraphViz, and gnuplot all producing SVG, with PNG fallback: http://e.metaclarity.org/142/rendering-tex-graphs-to-svg-in-wordpress/

 **Andrew Stacey** · 1 November 2009

1. Discussions like this are wasted in blog comments, even threaded and with ratings like this one. But I've given up on trying to persuade people to join in a forum discussion on this so I'll record my comments here and keep cutting-and-pasting them wherever this issue gets thought about until finally someone actually does something.

2. Standards are incredibly important, and as mathematicians we should realise that. Imagine a proof that said: "… Thus we have a regular Hausdorff space. Urysohn's metrisation theorem says that a separable regular Hausdorff space is metrisable. Obviously what he meant to say was that any regular Hausdorff is metrisable so we can assume that our space is metrisable….". I recently learnt that some versions of IE will ignore the content-type header tag and look at the file to see what it thinks it is. This means that it is impossible to send a plain text file that might have html tags in it and be sure that the end user's browser will treat it as plain text. As users, the reason for standards is so that we can be reasonably sure that the end user will see what we meant them to see when we wrote the page. For a cautionary tale on that, read the post Opera and MathML on Jacques Distler's blog.
If I were in charge of a large website serving up mathematical content, I would put in a check for the user-agent and if it detected a non-compliant browser, then the site would either refuse to load or put up a message saying "Warning: you are using a broken browser. What you read may not be in any way related to what we wrote.". It's probably a good thing that I have nothing to do with such a website.

3. The server-client model in the web takes the TeX paradigm to its extreme. TeX helps us separate content from presentation and allows us to concentrate on content first, and then on presentation afterwards. Serving up webpages takes that one stage further by making it impossible for the author to know exactly how the page is going to be rendered. They therefore have to concentrate on making sure that "There is no possibility that they be mis-understood.". This separation is good, because it means that people with accessibility issues – or even just a preference for green text on a cyan background – can modify what they see to suit themselves. This is an issue I'm increasingly aware of. Inline images are just the worst solution, and doing things like making the background transparent or putting the original LaTeX in the alt tag are just patching holes in the deck when there's a great big huge one down below the waterline.

4. For dynamically converted content, a full TeX-to-whatever converter is always going to be a bad idea. One of the strengths of TeX is its ability to adapt to the author's style. Some of my style files are longer than some of my papers! Asking for dynamic true conversion is a huge load. On the other hand, dynamic true conversion is the wrong answer: that's the sort of answer we'd expect from a company that produces huge software designed to do absolutely everything in one package (unfortunately, that comment no longer singles out one company). The True Path says "Do one thing, and do it well.". What is really wanted is static true conversion, in which case there are converters like tex4ht or plastex, and dynamic basic conversion, in which case there are converters like itex2mml or blahtex.
In addition, if you want cut-and-paste then you definitely don't want dynamic true conversion. I like to redefine \\R as \\mathbb{R}. So if you cut-and-paste from one of my documents you also have to cut-and-paste from the preamble. Except that it isn't in the preamble, all you get in the preamble is \ \usepackage{mymacros} (and it may be even more complicated than that).

5. What is really needed is for the heavyweights, such as our esteemed host here, to throw their weight behind MathML and to push the people behind the STIX fonts to Get A Move On! My university won't install the STIX fonts system-wide until they've been officially released which is Extremely Annoying. Mathematicians are a minority (which is another reason why we should be especially aware of the needs of other minorities), but one that can punch above it's weight. But we need the heavy-hitters to get into the fight.

As well as the above, I agree with everything that Jacques Distler said (but then I would say that, wouldn't I?).

 **Peter Krautzberger** · 2 November 2009

I know that the following is not exactly the topic of this discussion, but I hope it's worthwhile to mention.

What I feel mathematics on the web lacks is not only a good technology to display mathematical notation. What I miss are ideas when it comes to new ways of "telling" mathematics using web technology. Most mathematical content on the web is designed as if copied from print. This is ok, if the print is a great read anyway, but is it enough?

An example from the natural sciences: It might be a bit late to try it out (now that it has become commercial), but http://www.scivee.tv is/was an interesting place to experiment with a fusion of text, audio and video.

New, dynamic presentations (e.g. rich math applications, as discussed by Leifer and Hecker above) are missing. Is it only due to lack of skill (and time)? Is anyone discussing ideas? (Pointers very welcome!)

Of course, this effects implementation of notation, too. As Andrew Stacey pointed out, the web needs to seperate content and presentation even stricter. One might add "storytelling" to this. New mathematical "storytelling" would have to find its place in the balance between display of notation, storytelling technique and flexible presentation.

>  **Jacques Distler** · 2 November 2009
>
> *"For a cautionary tale on that, read the post Opera and MathML on Jacques Distler's blog."*
>
> Here's the link.
>
> *"What I miss are ideas when it comes to new ways of "telling" mathematics using web technology. Most mathematical content on the web is designed as if copied from print."*
>
> MathML is, of course, stylable with CSS, and fully scriptable, with Javascript. If you want dynamic content, that would be the way to go. Currently, there's not much support for dynamic content in itex2MML, though I'm open to suggestions.
>
> The only thing that is currently supported is turning individual terms in an equation into hyperlinks, using itex's \href{}{} command. There are other things which could easily be implemented, such as \toggle{}{}, if there's interest.
>
> Is there interest? Other suggestions?
>
> >  **Andrew Stacey** · 2 November 2009
> >
> > I think that you are right, but you don't go far enough. It's not that there isn't a technology for better presentation but rather that there aren't any ideas! I've asked in various places for examples of good presentation of mathematics on the web but so far I have to say that I'm a bit disappointed with the answers. All I've seen so far is pretty much taking something for print and putting it on a screen. But I'm not a graphic designer nor someone with great experience of different presentational styles so I don't know what the answer is, or even a half-step along the way.
> >
> > So anyone reading this who knows of a good website that presents mathematics in a different and good way, please let me know!

 **Robert Miner** · 2 November 2009

Returning for a moment to the technical question of rendering mathematics easily and effectively across a wide-range of browsers, I want to point to the MathJax project (www.mathjax.org), which is highly relevant here.

MathJax is a joint project of Design Science, AMS and SIAM with support from APS and other organizations. The goal is to develop an open source, JavaScript-based math display software. Davide Cervone is the lead developer. Davide also developed the jsMath package that has already been mentioned here. So one can think of MathJax as building on the ideas of jsMath, taking advantage of the intervening 5 years of browser technology development, and with the stability and reach of a sponsored project.
Key points of MathJax are that it will render
– both MathML and LaTeX in an HTML page
– it will have a rich, modular architecture
– it works well in the vast majority of browsers including all the main ones
– it requires no installation of anything on the part of the user
The project is aiming for an initial release by the end of the year.



Terence Tao · 2 November 2009

Regarding conversion between LaTeX and a web format (let's say MathML for sake of discussion), it's true that many "global" features of LaTeX, such as equation numbering or macros, would not work well with web formats, particularly if one wanted to cut-and-paste. However, one could imagine a paradigm like this:
Full LaTeX <—> reduced LaTeX <—> MathML
in which one first converted a regular LaTeX file into a file using only a limited subset of LaTeX (closer to original TeX, actually), by automatically expanding out macros, freezing equation numbers, etc., and then have full transferability between text in this reduced LaTeX format and the web format (by encoding the source LaTeX in the MathML by the analogue of the image ALT text in the image rendering model). It would still have some wrinkles (e.g. moving theorems and equations around on the web with cut-and-paste would mess up the equation numbering), but it would still allow things like one-click publishing of LaTeX files as web documents (by composing two of the above arrows), and conversely importing web documents as a (rather ugly) LaTeX file. (One may have to work a bit though to ensure that the LaTeX file captures other aspects of HTML markup properly – one may need a special mathml.sty file on the LaTeX side, for instance.)



Andrew Stacey · 2 November 2009

I guess this is what I really don't understand: why would anyone want to do this? It's a really difficult thing to do completely (even tex4ht doesn't get it completely right) and I'm not convinced that it's the right problem to be solving.
When I write for the web I write in a completely different way than when I write a paper. Papers can be long and technical, but stuff on the web tends to be shorter and snappier. When writing for the web then I can handle using a downgraded LaTeX like iTeX, but when writing papers then that quickly becomes extremely irritating and that's why my style files end up bigger than the papers!
What I really want is for the differences to be minimal so that I don't make stupid mistakes when changing from writing for one place to the other. Yes, I suppose I might want to cut-and-paste the odd little bit, but if doing a lot then it's an indication that I'm being lazy and that I'm not really thinking about who I'm writing for.
After all, one easy solution would be to make TeX itself the "browser" and send LaTeX files back and forth. It's easy enough to ensure that local style files are copied with the source and so long as everyone has an up-to-date version of TeXLive, all the global style files would be the same. Then anyone who needs to overlay styles for the sake of accessibility or just preference can do so and no-one has to worry about translating from one mark-up to another. Really, the idea that everything has to be done "in the browser" or "in the cloud" is simply ridiculous.



Terence Tao · 2 November 2009

It's true that for the type of maths that we currently do on the web, one wouldn't need much additional functionality (after all, if we are currently doing it, then the current functionality must already suffice!). But having more compatibility between LaTeX and web formats would allow for a number of new ways to do mathematics online that are not even being attempted right now, due to the limitations of existing tools. For instance, one could convert the entire preprint collection on, say, the arXiv, to a non-ugly web format, which could then be hyperlinked, commented on, excerpted from, indexed and searched, aggregated, and edited in a way which isn't currently convenient (though not utterly impossible) when one has to work with static PDF files or LaTeX source. Or, one could collaboratively work on a LaTeX paper using an off-the-shelf collaboration tool (wikis, Google Wave, etc.). (Our current attempt to collaboratively write up the results of our Polymath projects, via a MediaWiki that has no support for raw LaTeX, is not exactly satisfactory.) Or, one could export the contents of a maths-intensive blog to a LaTeX or at least a PDF file (I've had several readers ask me for this type of functionality, and I have no idea how to do it easily). Or, when running an online reading seminar over a single paper as I am currently doing on this blog, one could be annotating the paper itself rather than continually referring to "Corollary 1.2 on page 18" and so forth. Or, one could automatically sync the latest draft of one's paper or monograph to some online format so that collaborators or colleagues can comment and perhaps even edit in real time. And so on and so forth.

Because I intend to convert my own articles into print format, I am currently using LaTeX as the format for the master copies, and using conversion tools to publish them here. It works reasonably well, except for the fact that updating the blog posts whenever corrections need to be done is now a bit more annoying than it used to be, and certain features of the web format (e.g. images and video) are not supported by the conversion tool. Having better tools in this regard could allow mathematical blogs to approach publication-quality levels more easily than is currently possible.

It's possible that one could get by just by making an effort to make the web math input format as close to LaTeX as possible, as you say, though once one gets to more sophisticated LaTeX elements (e.g. tables and figures, bibliography, labels, etc.), or when trying to publish something too large to fit on a single web page (e.g. a full paper or book), I wonder if one can actually display such elements properly without either a full LaTeX->web conversion tool, or else a whole new language one would have to learn in addition to LaTeX. Of these two options, I think most people would prefer the former, if they had a choice.

[In any event, most of the things on my wish-list above are probably too difficult to accomplish in the near future; I'd be happy for now with a convenient way to display snippets of maths on the web that improves upon the current server-side image-rendering model.]



Andrew Stacey · 3 November 2009

(With apologies for the horrendous formatting, I have no idea how this blog formats its comments and there's no preview for me to try out the obvious things.)

There is an implicit assumption here that "in the browser" is the Right Way to do things. This, I find daft. Browsers are for viewing things, with a limited amount of interactivity possible. But for real interactivity then you need a local copy that you can work on which can then be synchronised with the master copy – you need a distributed version control system. For example, compiling my lecture notes already takes of the order of a minute on the super-duper-fast machine in front of me. If I had to send it off to a remote server,

wait for it to compile there, and then download it again, I'd get so annoyed I'd probably throw the computer out of the window!

That aside, everything you ask for can be done with the current technology.

1. Converting the entire arXiv: firstly, what's ugly about PDF? It can be hyperlinked, it can be scaled (better than HTML+Images!), you can be sure that you are reading what the author intended, just about every computer on this planet can read it. But nonetheless, for one-time conversions you can use tex4ht. It's pretty good, and is probably as good as you are going to get for a true TeX-to-other-markup-language converter (though I've heard rumours that the forthcoming luatex will be able to do this and more).
2. "hyperlinked, commented on, excerpted from, indexed and searched, aggregated, and edited". PDFs are hyperlinkable. As for comments, why would you want the comments embedded in the document? That would very quickly get unreadable. Rather you want a separate system that aggregates the comments so that you can read the original article in one window and follow along with the annotations in another. Even putting links in the document to the comments would quickly get irritating so all you'd really want is to be able to turn a page simultaneously in both applications – trivial. PDFs can be searched, and if not you could always search the original source text since that's available from the arXiv. In short, all of these things are really meta-information to be added afterwards and thus separately.
3. "work collaboratively on a LaTeX paper". That can be done with current technology. All I need to do is tell you where the main repository for my paper is kept and you can download a copy, work on it, and then merge it back into the main tree.
4. "Our current attempt to collaboratively write up the results of our Polymath projects, via a MediaWiki that has no support for raw LaTeX, is not exactly satisfactory." All I can say to this is that you should have been using Instiki (but, of course, I would say that).
5. "Or, one could export the contents of a maths-intensive blog to a LaTeX or at least a PDF file." Going FROM (X)HTML(+whatever) is always going to be easy because it is only a markup language. The difficulty is in going in the opposite direction because TeX is not just a markup language, it's an entire programming language.
6. "one could be annotating the paper itself rather than continually referring to "Corollary 1.2 on page 18" and so forth." This comes under the heading of meta-information again. Any long annotations are going to get irritating and interrupt the main flow. All you really want is to be able to link from the original document to the notes that are relevant near that section. That's simple.
7. "Or, one could automatically sync the latest draft of one's paper or monograph to some online format so that collaborators or colleagues can comment and perhaps even edit in real time." That's called a version control system. This is trivial with current software. The only thing that makes it remotely complicated is if you insist on doing it through a browser.

Here's the workflow for a DVCS:

1. Start a project on some public server (a sort of mathematical sourceforge – it's no coincidence that I registered the name 'mathforge' for the host for the n-lab).
2. Each collaborator gets access to the project and, using a DVCS, can download the latest copy of the master document, work on a branched copy, upload changes, merge changes from another author, convert to whatever format they like …
3. When two collaborators want to work together, they have a choice. If they want to work on something actually mathematical, they fire up something like jarnal which allows them to have a shared virtual whiteboard which they can write on – after all, if I want to think about maths then the last thing that I want to worry about is what symbols

I should use. This, combined with a VOIP system, means that they can talk and write just as if they were at a board together. If they want to work on a document, they fire up something like gobby which allows them to do real-time editing of the document together with real-time chatting (or use the VOIP again).
Back to your comments:
8. “Because I intend to convert my own articles into print format…” I have two, sort of contradictory, responses to this. Firstly, this means that you are writing your articles for two different media which means that you are never going to be fully happy with either. Either you can’t fully exploit the flexibility of the web (so no animations, no interactivity), or the print version is going to be a shallow imitation of the blog – let’s face it, the “certain features of the web format” that you refer to are never going to be properly supported in print format, let alone by the conversion software! So do you really want to do this? My other response is that your difficulty is self-imposed by using wordpress (well, to be honest I don’t know wordpress that well so it may be possible to do this via wordpress). My entire website is actually a blog, though you wouldn’t know it. I write text files in whatever format I like, I place a comment in the file as to what format I chose to write it in: LaTeX, markdown, multimarkdown, babytex, whatever, and then the blog software converts it appropriately. The text files are in a version control system so I can easily make changes, updates, roll-backs, and the blog software can also convert to other formats if so desired.
9. “It’s possible that one could get by just by making an effort to make the web math input format as close to LaTeX as possible…” Have you looked at the format produced by tex4ht? If you want an example of a really long, document, look at http://www.math.ntnu.no/~stacey/Seminars/mathml/noframes/diffloop_nocd.xml. This was a tex4ht conversion on a long (54 pages) document. The style is nothing to write home about, but that’s just a matter of putting a decent CSS over the top, but the conversion is absolutely fine. And it’s split it nicely into sections according to the sections in the original document. Or one could use the markdown+itex language that the n-lab uses. It’s not too different to what you’re doing now. The mass of mathematical bloggers out there don’t seem to have a problem with learning markdown syntax and, as the n-lab pages demonstrate, one can do just about anything in that language. The only thing we’re having issues with is really complicated diagrams, and that’s more because there isn’t a proper way to mix MathML and SVG yet, and because it’s easy to type a paragraph but crafting a picture takes a little more time.
Finally: “I’d be happy for now with a convenient way to display snippets of maths on the web that improves upon the current server-side image-rendering model.”. Then throw your weight behind MathML. If you converted your blog to MathML then the backlash would be immense. But it wouldn’t be a backlash against you, it would be against those who keep dragging their feet over the STIX fonts, those who don’t assign any resources to fixing bugs in MathML implementations, those who don’t even have any MathML implementations(!).



Michael Whapples · 3 November 2009

I think Andrew raises some good points. In the main I think material which is primarily aimed at the web will be different to that primarily aimed for print. One time where putting the print version online is if you wish to make a print document available online. Unfortunately I don’t think PDF is good for this from the angle of accessibility and the web is such a good chance to make documents accessible. PDF has the potential to be able to make maths accessible but they require some work and tools would need

to be altered to produce the required tagging information. MathML is about the best option, and remember tex4ht only works on the source LaTeX so I am saying don't just publish a PDF.
Now for web only content, like Andrew says don't drag your feet. I say unless more authors switch to MathML and so require their users to cope with MathML then progress will be slow and the take up of MathML has been slow enough. Equations in images aren't really a accessible solution and there is an accessible alternative (which also brings other advantages to other users).
I know I might be banging on about accessibility, but its such a fustrating situation to be in. If you find it hard to find decent math support on the web, well imagine adding to that trying to find an accessible one. Also just to make it more fustrating, I know its not due to the technology/standards being there, we do have a solution.
OK, moving on from that, there is the question of math input, then I think a TeX like language (this is what I am most familiar with, if others can think of an alternative then may be that) which could be used to enter equations. When I say TeX like I don't mean full TeX, may be itex would be suitable. Input like this would pose little in the way of accessibility issues. Again I am thinking like Andrew that web only content is probably not going to have a huge amount of maths input and so full LaTeX is not required.



Andrew Stacey · 3 November 2009

(This is really a reply to Michael's comment but that level of nesting isn't allowed – which brings up another issue: there's some important points being made here that could well get lost in blog comments)
I'm interested in what you say about PDF. My natural assumption would have been that PDFs were alright – not ideal, but alright – as far as accessibility is concerned since they seem fairly scalable, but maybe they aren't really or that's still not enough.
Is there somewhere that I can learn about the issues involved with accessibility? Especially visual accessibility? I'm quite keen to learn about this.



Jacques Distler · 3 November 2009

Andrew:
Here's an old blog post of mine about PDF accessibility. And here's a followup that I wrote a year later. Lots of links therein.
Since then, I don't think much has happened. PDFTeX still doesn't create tagged PDF — the minimal benchmark for accessibility. And we're very far from creating accessible equations (via embedding MathML).
Michael:
Since you've a vested interest, I'd like to invite you to take a long hard look at the accessibility of Instiki. Any suggestions as to what can be done to make the application more friendly to the visually-impaired would be much appreciated. (If you feel like diving into the source code, that would be great, but certainly not required.)



Terence Tao · 3 November 2009

It's true that browsers inherently have limits to their interactivity, though the situation seems to be improving with time. For the purposes of online mathematical collaboration with mathematicians who are not necessarily computer-savvy, though, it really seems to be the only practical option available, basically because the browser is almost the only piece of sophisticated software with the required levels of functionality that most people are willing or able to use.

I could see myself, if I had to, go download some external software, compile it, configure it, learn how to use it, and deal with whatever glitches that occur with it, and I am sure many other mathematicians with the right kind of computer skills could easily do so also (and probably more efficiently than I, actually). But, a large fraction of my collaborators would not do this unless it was something that was already in mainstream use (like email, browsers, or LaTeX editors) that they would have a significant incentive to learn how to use outside of their collaboration with myself. The barrier to entry can't be significantly higher than, say, registering an account on a web site (and even this causes trouble sometimes), or else most people will abandon the effort at the first sign of difficulty (e.g. the need to update a driver, or something). Any "solution" to an online collaboration task that requires all collaborators involved to first perform a five-step download, installation, configuration, and registration process for an unfamiliar and non-standardised piece of software (which may well become obsolete or in need of upgrading in a few years anyway) is basically a non-starter for a typical mathematical collaboration, no matter how "easy" or "trivial" this process would be to a tech-savvy person. [Also, in some cases, one may not even have permission to install new software on one's work computer, or have a computer whose OS is not supported by the software.] It may work great for a small minority of early adopters, but will never be mainstream. (Though perhaps if we filter to the subset of mathematicians who already use blogs, the percentages might be better.)

Incidentally, this is one reason why I stick with the wordpress.com hosting service, despite its various drawbacks (lack of comment preview and editing, for instance); it's essentially trivial to set up a blog here and run it without any need to be a sysadmin or code maintainer, and indeed dozens of mathematicians have done so already. I am sure that with a customised blog and a certain investment of time and computer skill, one could have a significantly better presentation; but this blog already absorbs all of my free research time as it is, and works well enough that the diminishing returns of further improvement are not worth the current level of time and effort required to migrate to a better platform, not to mention the disruption to readers. Now if all I had to do was check a box marked "Enable MathML", or even something slightly more sophisticated such as configuring my CSS, it wouldn't be an issue; but it's nowhere near this easy at present. I'm willing to go through a five-step installation process, but I do not want to spend significant amounts of time trying to install code patches or configure hosts every few months.

Incidentally, I agree with you regarding metadata on existing LaTeX or PDF documents; I wasn't suggesting that a web copy of these files becomes the master copy, and some sort of overlay where one could easily jump back and forth (or expand and collapse, or hover and reveal, etc.) between the source and the commentary would be fantastic. I am already very happy with the way PDF-compiled LaTeX can contain internal crossreferences (e.g. clicking in a reference to Theorem 3.4, and being transported to the statement of that theorem); but currently I can't externally link to those references from a web page or from a different PDF document. An overlay could be a good solution to many of the things I have in mind.



Matt Leifer · 4 November 2009

Hmm, I've had a few email exchanges with Andrew about these issues and the basic difference is that he doesn't seem to buy into the whole web2.0, cloud computing fandango as much as I do. In my view, the distinction between "writing for the web" and "writing for print" is a big red herring. The web is a big place and there is room for

putting a variety of content types on it, both long and short form. Also, I think that the distinction between “writing for the web” and “writing for print” will disappear in the near future, particularly as ebook readers get more sophisticated and widely adopted. Turning blogs into books is already quite common, which is something that I should not need to mention on this particular blog.
I agree that distributed version control systems are a great solution, but they are not for the faint of heart. A good collaborative tool should offer straightforward web interfaces for the average user, whilst allowing technically inclined users to leverage the full power of a DVCS.

 **Andrew Stacey** · 2 November 2009

Oh, and macros aren’t hard to do either. My attempt at doing a LaTeX-to-something converter (PHPLaTeX) handles macros just fine. The problem is that as soon as you introduce macro handling then you introduce the potential to seriously effect the compilation speed and in a dynamic situation, that’s not acceptable (in a static once-for-all situation then it might be alright).

 **Jacques Distler** · 2 November 2009

“it’s true that many “global” features of LaTeX, such as equation numbering … would not work well with web formats”
Automatic equation numbering works just fine.
Instiki even does automatic numbering of Theorems, Lemmas, etc.
Macros are a bit harder to implement (in a fashion that makes sense on the web, is efficient, etc). A primitive macro facility is certainly on my TODO list, but don’t hold your breath.

web math « Sllu's Blog · 2 November 2009
[…] post summarizes the article by Brian Hayes on American Scientist: writing math on the web. See also live discussions at Terry Tao’s blog, STIX fonts, itex2MML and […]

 **Michael Whapples** · 3 November 2009

Here is a post to the NFB blindmath list which discusses the current situation of math accessibility in PDF http://www.nfbnet.org/pipermail/blindmath_nfbnet.org/2009-October/002226.html
OK, may be this isn’t too revealing about the technicalities of math accessibility of PDF, but it does give an idea of where we currently stand. It may be worth you contacting Neil Soiffer for more details as I believe he has done most of the work on it up to now.
I will just finally note that math accessibility should be considered separately to general accessibility of PDF, for non-mathematical documents I believe there are solutions which can add tagging.

 **Michael Whapples** · 3 November 2009

Jacques, instiki looks good from my angle as a screen reader user (well at least the little I have tried out). My only issue was not being able to read mathml in firefox on linux with the orca screen reader (that’s an issue for firefox and orca) and as I don’t like using cygwyn it meant having both a linux machine (to run instiki) and a windows machine to view the mathml with mathplayer. Although the mathml is a problem on linux when using orca, I had no problem editing pages and viewing the source. I will ask some who have low sight but do use their vision to have a look at instiki.

 **Andrew Stacey** · 3 November 2009

(This is in reply to Terrance Tao’s last comment, but the thread level is too deep.)
To a certain extent, that’s the killer blow: if people aren’t willing to actually make use of what’s available then there’s no way to force them. However, just because someone else refuses to use (or,

let's be charitable, can't use) a proper system doesn't mean that I have to tie my hands behind my back and type with my nose!

I actually think it would be quite easy to design a system that had a VCS backend located on some central server with clients that worked locally. It could work over a web protocol, or other, so those without permission to install software could work through a browser but those able (and willing) to install the VCS could have all the benefits that that would entail.

The point about a VCS is that it gives people the choice to do things how they want. Those who want to be able to write papers with ease and without always feeling that they are fighting their computer can use a decent text editor, whilst if someone really wants to use Word then they can.

I absolutely hate these little text boxes in browsers. I am a commandline junkie, and have lots of shortcut keys for lots of different programs, and muscle memory says that if I'm typing lots of text then I must be using Emacs so I keep hitting things like Ctrl+N or Ctrl+T when I'm using one of these text boxes. Fortunately, there's an addon called "It's all text" which allows me to actually use Emacs to edit one of these text boxes! When I'm editing a page on, say, the n-lab then I do that: I use the "It's all text" addon to link the text box to an Emacs session, edit the page in Emacs, I even can do "compile-preview" cycles on my local machine (i.e. without sending it to the remote server), and when I'm happy with it then it automatically gets put back in the text box and I can submit it as usual (with the added bonus of having a local copy if it all goes wrong).

I'm not saying that everyone should do that, but that everyone should have the choice of doing that if they like. But putting it all in the browser first is backwards, what I've described above is much more of a complicated set-up than it would need to be if the browser bit was secondary to the version control system.

On to another, but related, point. You say you use wordpress so that you don't have to worry about being sysadmin. But that sysadmining still has to be done, you've just farmed it off to the wordpress guys. I'm not suggesting any change in this, just that the centralised system be set-up with scientific collaboration in mind rather than Joe Blogs's Blog. I don't know, but I would suspect that it would be hard to adapt a wordpress blog to deliver MathML because of the strict nature of the doc-type (though what I've read here and there of how HTML5 will work is encouraging). Imagine a system where you could simply click a button to turn on MathML but where you didn't have to do any sysadmining because some Nice Bloke was doing it all for you.

After all, when you use Firefox, Chrome, Google, or even MS Word, then you're not using a static product. Every now and then a little box comes up saying "It's time to download the latest version". Some systems hide that away, others make a feature of it. But it's there nonetheless.

I don't mind delving into the technical side of things. I'm not a programmer (even an amateur one), I'm a bit of a hacker, and I know enough to keep a system running. I like being sysadmin for the n-lab – I like knowing that what I do there helps others do their maths. Sometimes, I even get to do some myself! What I really like is when they don't notice that I'm keeping it running for them. I don't want them to feel like they're always having to adapt to changing circumstances, but that the system works well and every now and then they notice that it seems to work better than before.

Similarly, while I like using complicated systems myself, I don't insist that my collaborators do the same. I send documents back and forth by email, but what I do is have a sort of "virtual DVCS" for my collaborators: they send me stuff by email and I put it into the VCS for them, and then merge it with whatever I've been doing in the meantime. But just because they can't or won't install a DVCS doesn't mean that I can't.

I feel I'm beginning to ramble. I apologise. I'll stop now.



Terence Tao · 3 November 2009

Having a web interface with a non-web backend would work quite well, actually. I would definitely switch to some blogging software, well integrated with LaTeX, that I could download

on my own computer, and which enabled me to publish online at the click of a button (or even have it sync automatically), especially if the sysadmin'ing could still be farmed out to another person. (Actually I do a crude version of this already with Luca's Python script to automatically convert LaTeX to HTML.)
Similarly with version control systems with an optional web interface. I already have encountered this sort of hybrid situation before; I run a (non-mathematical) mailing list in which some users have set up a web account to use the full features of the list (e.g. collaborative editing of documents), but others haven't and can only use the email alias for the list, as well as the URL for the archives of past discussions and uploaded files. It's a reasonable compromise between having least-common-denominator functionality, and requiring everyone to have a certain level of technical competence.
A company or organisation that offered to host scientific blogs and wikis that supported and developed the type of features we would like to see (e.g. mathML), and was easy enough to use by the typical scientist (i.e. at the level of clicking a button), could indeed be a major advance. Somehow though I don't think we have the market power or manpower to make this happen (though perhaps other sources of funding might be available for a project like this). Especially given that the *ad hoc* use of off-the-shelf tools often is "good enough" for most purposes, even if less than ideal at times.



hecker · 3 November 2009

"A company or organisation that offered to host scientific blogs and wikis that supported and developed the type of features we would like to see (e.g. mathML), and was easy enough to use by the typical scientist (i.e. at the level of clicking a button), could indeed be a major advance."
Mozilla would not be willing or able to run such such a service on an ongoing basis. However there's a possibility we might be interested in working with others who'd like to prototype such a system based on modern web technologies. Our involvement would be similar to what we're doing with our current Processing for the Web project: We would endorse and help promote the project, would assist with project coordination and management to some extent, would help recruit students and others who'd like to work on it, and where and when appropriate would have Mozilla developers provide advice on using Mozilla/Firefox technologies and fix any Mozilla/Firefox bugs.
From our point a project like this could be a good advertisement for the new web technologies being implemented in Firefox and other modern browsers, and could help serve as a forcing function to motivate us to improve our implementation of such technologies and our support for MathML, etc.



jonathanfine · 4 November 2009

Terry wrote, in his original post: " 5. It is difficult to take an extended portion of LaTeX and convert it into a web page or vice versa, although tools such as Luca Trevisan's LaTeX to WordPress converter achieve a heroic (and very useful) level of partial success in this regard."
There are standard, reliable and widely used tools for parsing and processing XML. For LaTeX there are no such tools. There are not, in my view, any such tools for LaTeX (although my favourite 'new-miss' is PlasTeX).
As Jacques Distler pointed out, there is William Hammond's Gellmu, which can be readily translated to XML, but that has not been widely adopted by the TeX community.
I'd say decent tools for converting X to XML, where X is LaTeX or Gellmu or something that authors are willing to use would be a big help in putting maths on the web.

This, of course, would not be a change in web standards but an adoption of a web standard (namely XML) by the TeX using community.

 **Matt Leifer** · 4 November 2009

There are quite a few wiki engines out there that are backed by a distributed version control system. This means you can easily edit them online using the browser, but also checkout the repository and work with the files in a local text editor. I am working on getting one of these wikis LaTeX and MathML enabled as a possible solution for collaborative paper authoring. I can't solve the hosting problem, but I intend to make the installation as painless as possible, including on cheapo shared hosting accounts.
Adapting this to a provide a simple engine should also not be too difficult, although it would not have all the features of WordPress.

 **hecker** · 3 November 2009

"I actually think it would be quite easy to design a system that had a VCS backend located on some central server with clients that worked locally. It could work over a web protocol, or other, so those without permission to install software could work through a browser but those able (and willing) to install the VCS could have all the benefits that that would entail."
The Mozilla Labs group is doing something similar to this as part of its experimental Bespin project. Bespin provides a rich web-based code editor implemented in the browsers, with code stored on a server using standard version control systems. (Bespin currently supports Subversion and Mercurial for version control.) You can interact with the code repositories using Bespin through the web browser or use locally-installed editors and VCS utilities. (If anyone is interested in trying Bespin out you can register for a free userid.)
We've proposed creating a Bespin-based system to provide a web-based development environment for our Processing for the Web project, and I could see an analogous system being used for creating and disseminating math-related content.

 **Andrew Stacey** · 3 November 2009

(I'm giving up on the threading, sorry, it's too complicated to follow this late at night)
Bespin sounds interesting. I just did a quick search on web interfaces to bazaar (my particular choice of DVCS) and was pleased to find that there were plenty to choose from. I may give that a spin, if you'll pardon the pun.
Regarding farming out to a company: No Way! Let's keep this in-house. We're the one's who'll use it, we should design it (whatever "it" is!). By all means, let's use their expertise, and so forth, but let's keep control.
A DVCS backend to a blog is a slightly different matter to what we've been discussing. That wouldn't be difficult to set up. All the pieces are there, it's just a matter of putting them together. The main question is what interface you'd want or be prepared to use. As I think I said elsewhere, my whole website is actually a web-frontend with a DVCS backend. It would need minimal modification to suit your needs, I think.
(But of course, if you just want a MathML-enabled blog then talk to Jacques Distler – his musings, and the n-category cafe, are such)

 **Jacques Distler** · 3 November 2009

"(But of course, if you just want a MathML-enabled blog then talk to Jacques Distler – his musings, and the n-category cafe, are such)"
In my copious free time, my plan is to port that software to Melody, the open-source (GPL) fork of MovableType.

Dunno exactly when that will happen, though. There's some chance I can get a summer student to work on it, which seems like the best bet, at the moment ….
"A company or organisation that offered to host scientific blogs and wikis that supported and developed the type of features we would like to see (e.g. mathML), and was easy enough to use by the typical scientist (i.e. at the level of clicking a button), could indeed be a major advance."
ncatlab.org seems to play host to about 20 Instiki wikis. Most of those are fairly small, though the main one has something over 2300 pages.
I'm not suggesting that they (which is to say, Andrew Stacey) play host to all and sundry; rather, I'm pointing out that it would not be too hard to set up one or more similar installations. (Andrew can speak to the hosting requirements; seem fairly run-of-the-mill.)
As to a blog/wiki with a DVCS back-end, I know people have written such. Personally, I'm rather spoiled, not merely in having an RDB back-end, but in having an abstraction-layer (ActiveRecord, in the case of Rails), which means that — as a programmer — I don't have to monkey much with raw SQL calls. Working with a DVCS backend would be a bit of a pain, in that regard.
But that's just me …



Matt Leifer · 4 November 2009

I don't think so. If you use a DVCS backend then you could ditch the database completely and just store all the content in flat-files. The DVCS would make sure that the history would be stored efficiently and interacting with a DVCS is similar to dealing with an abstraction-layer.
Anyway, I hope to come up with a proof of concept of this within a few months.



Jacques Distler · 4 November 2009

Take a very *simple* example.
1) in many wiki systems (including Instiki), it's possible to include the contents of wiki page X in wiki page Y.
2) To speed delivery of pages, there's a cache of already-rendered pages.
Say I edit page X. I now have to expire the caches for all the pages (among them, Y), which include X. How do I know which pages to expire?
With a RDB backend, that's a trivially simple SQL query. With a DVCS backend, it's not.
There are many, many similar bits of functionality which, broadly speaking, have to do with metadata about the pages in the wiki. (What are the pages authored by a given author? Which pages link to a given page? What are all of the pages in the category "Spoons" which have no inbound WikiLinks? …)
Of course, one can arrange to implement this functionality, by storing this metadata in the DVCS (essentially, reinventing one-by-one the features of an RDB).
But that's not a very pleasant prospect, now is it?



Peter Krautzberger · 3 November 2009

(I follow Andrew's idea to ignore threading. I apologize for the length.)
This comment is an addition to my first comment, a reaction to Andrew's first paragraph here http://go2.wordpress.com/?id=725X1342&site=terrytao.wordpress.com&url=http%3A%2F%2Fwww.math.ntnu.no%2F~stacey and a semi-spontaneous response to Jacques' tempting "any other suggestions?". I hope this explains a bit what I meant with "storytelling" as an element of displaying mathematics.
My initial thought to Andrew's paragraph was: oh, the only thing PDF is good for is printing. Unless you have a good tablet pc with Xournal/Jarnal etc. nothing beats reading a PDF printed out. To quote Andrew: This, I find daft. Reading text on the web works differently and for mathematics there is no

real attempt to adapt to this (although I loved Timothy Gowers' current experiment until I lost track of things).
The web is not a book (or paper). It cannot be(at) a book. When I read a (mathematical) book, I usually have all 10 fingers in the book, keeping some references in check, telling myself what to read next etc. I also browse back and forth, looking for keywords or sometimes just structural elements that tell me I have a chance to find whatever I'm looking for. Above all, I also know how much more there is to read before the current thought is completed, the proof, the example or the chapter ends (and I can relax and get a cup of coffee). All this I cannot do efficiently looking at a PDF onscreen. I admit, this is probably my fault — I grew up reading books, my bad (luck) — I also cannot afford (to carry) a fancy 27" screen to display multiple pages of a PDF simultaneously.
This is where Andrew's comment comes in: keeping content and presentation even stronger apart. Can we "display" mathematics in such a way that (besides being more accessible, which is the first and foremost issue) it displays _sensibly_ on any device — not "correctly", sensibly. Is a mathematical blog readable on an iPhone? What would it take to be?
I think it would require a new structural element (what I chose to call "storytelling"), a structural element that scales with the screen/device. Something that will help the reader e.g. by "telling" how much more before there is a pause, a break, a section, a chapter — no matter what device is used. And this structure would have to scale very well — and be open to client side modifications, too.
This structural element might be in the line of Uri Leron's wonderful ideas http://go2.wordpress.com/?id=725X1342&site=caicedoteaching.wordpress.com&url=http%3A%2F%2Fwww.jstor.org.libproxy.boisestate.edu%2Fstable%2Fpdfplus%2F2975544.pdf : display mathematics so that you can "zoom in" — give all details, but allow the reader to decide how much detail is displayed/requested, allow for only reading "the big picture", take Leron's "elevator".
It might also be a cover flow kind of presentation of a "normal paper", allowing for the natural browsing, annotations, links etc. while somehow ingeniously allowing readers not to loose themselves when they have to magnify the current page (or just scroll) (Andrew, can you make that happen with ajax and the tex4ht version you linked to?)
This structure really should depend on what you're telling and what is the best way to tell what you're telling (e.g. expository vs. logical proof vs. geometrical argument) — and you might even want to add audio (think: director's comments on a DVD) or even video (animation or just the talk or conversation this came from).
As I said, just some vague ideas.



jonathanfine · 4 November 2009

Terry – This post is getting quite long. It would be great if you could summarize and start a new one (and perhaps remind us of what the key issues are for you).



Displaying maths online, II « What's new · 4 November 2009

[…] | Tags: html, mathematical formatting, MathML | by Terence Tao As the previous discussion on displaying mathematics on the web has become quite lengthy, I am opening a fresh post to continue the […]



Simon · 22 December 2009

To add to Robert Miner's comments above, MathJax has now released a preview page:

> Global wellposedness for the Maxwell-Klein-Gordon equation below the energy norm

Simon

 **chris** · 27 September 2010

Maybe there's a bit of a solution for some of the problems listed in this blog?
Please visit one of my pages and have a look at **MathEL, our Mathematical Authoring Language for the Web** and how it works even in the WordPress environment.
How remote MathEL translation works

 **Link Starbureiy** · 23 November 2011

Terry-
I'm uncertain if I'm late with this or not, but has anyone reviewed the new Annotum tool? From the looks of it, it's simply a WordPress theme designed around PubMed specifications that lets publishers export as different formats (keeping, of course, an XML source). It also offers citation capabilities. What's surprising is that Google, Inc. is a co-developer. Anyway, this looks like an answer to the question of how to collaborate on science using the Web; as we all suspected, it would be incubated from a blog.
Feeback from the community is appreciated!

Mathematics on wordpress « __cs_waffle · 15 December 2011
[...] math formulae is still not well supported by the current html standards. (Terry Tao started a discussion on his blog about this.) MathML is an effort to extend the standards to support math forms but [...]

Публикация математических уравнений при помощи LaTeX | ewgeny · 14 August 2012
[...] [1] displying mathematics on the web [2] latex support Like this:LikeBe the first to like this. This entry was posted in [...]

Latex on the web | Zintegra · 15 January 2013
[...] https://terrytao.wordpress.com/2009/10/29/displaying-mathematics-on-the-web/ [...]

Resources for getting maths on to the web | CL-UAT · 27 December 2014
[...] thing that came out of Terry Tao's recent blog posts on this matter (first post and follow up) is that it's hard to get an overview of all the different ways of getting [...]

 **Dr. Sikun Lan** · 20 December 2017

Terence,
Just read your comments you wrote 8 years ago I am doing research on this subject for Geffen Academy at UCLA and for my tutoring business. I am wondering if you see any progress in standardization or general consensus up to this point.
Thanks,
Sikun